

Тема 2. Общие понятия реляционного подхода к организации баз данных. Основные концепции и термины

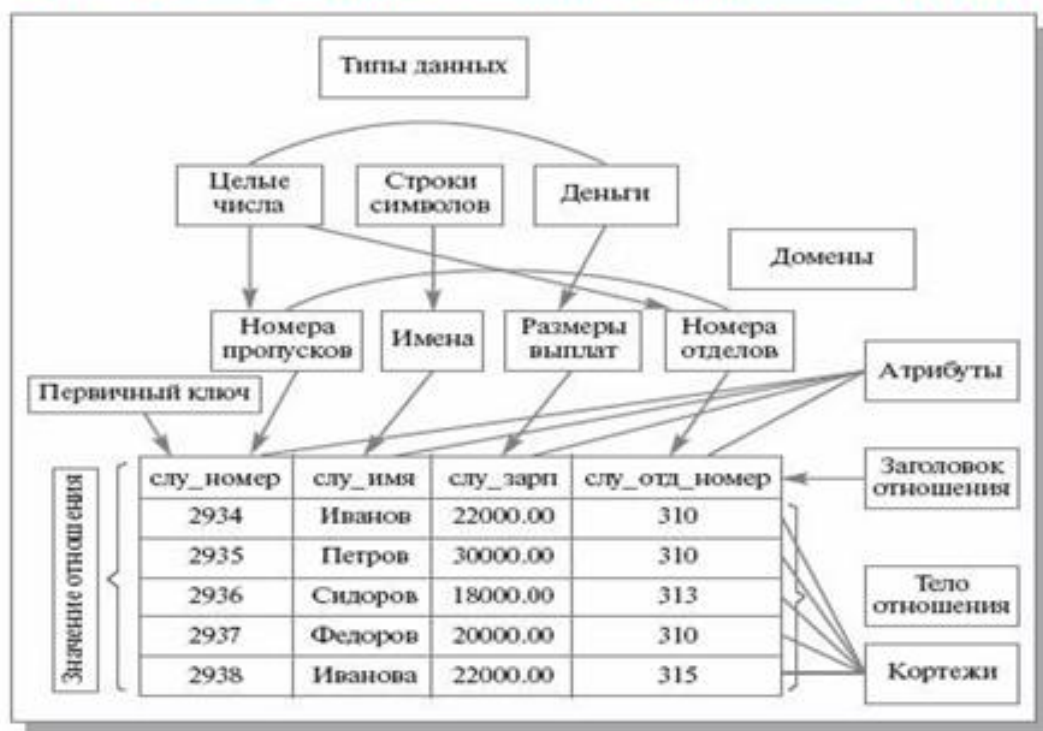
Основными понятиями реляционных баз данных являются: тип данных, домен, атрибут, кортеж, первичный ключ и отношение.

Тип данных. Понятие тип данных в реляционной модели данных полностью адекватно понятию типа данных в языках программирования. В современных реляционных БД допускается хранение: символьных,

- числовых данных,
- битовых строк,
- специализированных числовых данных (таких как "деньги"),
- а также специальных "темпоральных" данных (дата, время, временной интервал).

Достаточно активно развивается подход к расширению возможностей реляционных систем абстрактными типами данных.

Соотношение основных понятий реляционного подхода



Атрибут - это свойство сущности. Например, сущность КНИГА может характеризоваться следующими атрибутами: автор, название, год издания, цена, издательство, число страниц. Конкретные книги являются экземплярами сущности КНИГА. Атрибут, который уникальным образом идентифицирует экземпляры сущности, называется ключом. Может быть составной ключ, представляющий комбинацию нескольких атрибутов.

Различают следующие атрибуты

1. *Идентифицирующие и описательные атрибуты.* Идентифицирующие позволяют отличить один экземпляр сущности от другого. Описательные атрибуты заключают в себе интересующие нас свойства сущности.

2. *Составные и простые атрибуты.* Простой атрибут имеет неделимое значение. Составной атрибут является комбинацией нескольких элементов, возможно, принадлежащих разным типам данных (ФИО, адрес и др.).

3. *Однозначные и многозначные атрибуты* (могут иметь соответственно одно или много значений для каждого экземпляра сущности). Например, дата рождения – это однозначный атрибут, а номер телефона – многозначный.

4. *Основные и производные атрибуты.* Значение основного атрибута не зависит от других атрибутов; значение производного атрибута вычисляется на основе значений других атрибутов. Например, возраст вычисляется на основе даты рождения и текущей даты.

5. *Обязательные и необязательные* (первые должны быть указаны при размещении данных в базе, вторые могут не указываться).

Для каждого атрибута необходимо определить название, указать тип данных и описать ограничения целостности – множество значений, которые может принимать данный атрибут.

Домен. Наиболее правильной интуитивной трактовкой понятия домена является понимание домена как допустимого потенциального множества значений данного типа. Например, можно ввести домен "цвет". Для предметной области "Правила перехода улицы" домен "цвет" будет принимать значения: "красный", "желтый", "зеленый". Никакие другие значения для данного домена СУБД не пропустит.

Домен - это семантическое понятие. Домен можно рассматривать как подмножество значений некоторого типа данных имеющих определенный смысл. Домен характеризуется следующими свойствами:

- Домен имеет уникальное имя (в пределах базы данных).
- Домен определен на некотором простом типе данных или на другом домене.
- Домен может иметь некоторое логическое условие, позволяющее описать подмножество данных, допустимых для данного домена.
- Домен несет определенную смысловую нагрузку.

Кортеж, соответствующий данной схеме отношения, - это множество пар {имя атрибута, значение}, которое содержит одно вхождение каждого имени атрибута, принадлежащего схеме отношения. "Значение" является допустимым значением домена данного атрибута (или типа данных, если понятие домена не поддерживается). Попросту говоря, кортеж - это набор именованных значений заданного типа.

Отношение - это множество кортежей, соответствующих одной схеме отношения. На самом деле, понятие схемы отношения ближе всего к понятию структурного типа данных в языках программирования.

Степенью (или -арностью) отношения называют число атрибутов в отношении.

Реляционной базой данных называется набор отношений.

Заголовком (или схемой) отношения называется конечное множество атрибутов некоторого базового типа. Схема отношения - это именованное множество пар {имя атрибута, имя домена (или типа, если понятие домена не поддерживается)}. Степень или "арность" схемы отношения - мощность этого множества. Схема БД (в структурном смысле) - это набор именованных схем отношений.

Обычным пользовательским представлением отношения является таблица, заголовком которой является схема отношения, а строками - кортежи отношения-экземпляра; в этом случае имена атрибутов именуют столбцы этой таблицы. Поэтому иногда говорят "столбец таблицы", имея в виду "атрибут отношения". Этой терминологии придерживаются в большинстве коммерческих реляционных СУБД.

Схема БД - это поименованная совокупность схем, входящих в нее отношений.

Телом отношения называется произвольное множество кортежей отношения.

Значением отношения называется совокупность ее строк и столбцов.

Мощность отношения — количество кортежей в нем.

Как видно, основные структурные понятия реляционной модели данных (если не считать понятия домена) имеют очень простую интуитивную интерпретацию, хотя в теории реляционных БД все они определяются абсолютно формально и точно.

Фундаментальные свойства отношений

Остановимся теперь на некоторых важных свойствах отношений, которые следуют из приведенных ранее определений:

- Отсутствие кортежей-дубликатов.

То свойство, что отношения не содержат кортежей-дубликатов, следует из определения отношения как множества кортежей. В классической теории множеств по определению каждое множество состоит из различных элементов.

Из этого свойства вытекает наличие у каждого отношения так называемого первичного ключа - набора атрибутов, значения которых однозначно определяют кортеж отношения. Понятие первичного ключа является исключительно важным в связи с понятием целостности баз данных.

Забегая вперед, заметим, что во многих практических реализациях РСУБД допускается нарушение свойства уникальности кортежей для промежуточных отношений, порождаемых неявно при выполнении запросов. Такие отношения являются не множествами, а мультимножествами, что в ряде случаев позволяет добиться определенных преимуществ, но иногда приводит к серьезным проблемам.

- Отсутствие упорядоченности кортежей

Свойство отсутствия упорядоченности кортежей отношения также является следствием определения отношения-экземпляра как множества кортежей. Отсутствие требования к поддержанию порядка на множестве кортежей отношения дает дополнительную гибкость СУБД при хранении баз данных во внешней памяти и при выполнении запросов к базе данных. Это не противоречит тому, что при формулировании запроса к БД, например, на языке SQL можно потребовать сортировки результирующей таблицы в соответствии со значениями некоторых

столбцов. Такой результат, вообще говоря, не отношение, а некоторый упорядоченный список кортежей.

- Отсутствие упорядоченности атрибутов

Атрибуты отношений не упорядочены, поскольку по определению схема отношения есть множество пар {имя атрибута, имя домена}. Для ссылки на значение атрибута в кортеже отношения всегда используется имя атрибута. Это свойство теоретически позволяет, например, модифицировать схемы существующих отношений не только путем добавления новых атрибутов, но и путем удаления существующих атрибутов. Однако в большинстве существующих систем такая возможность не допускается, и хотя упорядоченность набора атрибутов отношения явно не требуется, часто в качестве неявного порядка атрибутов используется их порядок в линейной форме определения схемы отношения.

- Атомарность значений атрибутов

Значения всех атрибутов являются атомарными. Это следует из определения домена как потенциального множества значений простого типа данных, т.е. среди значений домена не могут содержаться множества значений (отношения). Принято говорить, что в реляционных базах данных допускаются только нормализованные отношения или отношения, представленные в первой нормальной форме.

Конечно, понятие атомарности довольно относительно. Так, строковый тип данных можно рассматривать как одномерный массив символов, а целый тип данных - как набор битов. Важно лишь то, что при переходе на такой низкий уровень теряется семантика (смысл) данных. Если строку, выражающую, например, фамилию сотрудника, разложить в массив символов, то при этом теряется смысл такой строки как единого целого.

Собственно, для реляционной модели данных тип используемых данных не важен. Требование, чтобы тип данных был простым, нужно понимать так, что в реляционных операциях не должна учитываться внутренняя структура данных. Конечно, должны быть описаны действия, которые можно производить с данными как с единым целым, например, данные числового типа можно складывать, для строк возможна операция конкатенации и т.д.

В реляционных базах данных допускаются только нормализованные отношения, или отношения. Нормализованные отношения составляют основу классического реляционного подхода к организации баз данных. Они обладают некоторыми ограничениями (не любую информацию удобно представлять в виде плоских таблиц), но существенно упрощают манипулирование данными.

В базе данных **первичные ключи** используются для идентификации. В жизни первичные ключи вокруг нас везде. Каждый раз, когда вы сталкиваетесь с уникальным числом это число может служить первичным ключом в базе данных (может, но не обязательно должно использоваться как таковое. Все базы данных способны автоматически генерировать уникальное значение для каждой записи в виде числа, которое автоматически увеличивается и вставляется вместе с каждой новой записью. Несколько примеров.

Номер заказа, который вы получаете при покупке в интернет-магазине может быть первичным ключом какой-нибудь таблицы заказов в базе данных этого магазина, т.к. он является уникальным значением.

Номер социального страхования может быть первичным ключом в какой-нибудь таблице в базе данных государственного учреждения, т.к. она также как и в предыдущем примере уникален.

Номер счета-фактуры может быть использован в качестве первичного ключа в таблице базы данных, в которой хранятся выданные клиентам счета-фактуры.

Числовой номер клиента часто используется как первичный ключ в таблице клиентов.

Что объединяет эти примеры? То, что во всех из них в качестве первичного ключа выбирается уникальное, не повторяющееся значение для каждой записи. Еще раз. Значения поля таблицы базы данных, выбранного в качестве первичного ключа, всегда уникально.

Что характеризует первичный ключ? Характеристики первичного ключа.

Первичный ключ используется для идентификации записей в таблице, для того, чтобы каждая запись стала уникальной. Еще одна аналогия... Когда вы звоните в службу технической поддержки, оператор обычно просит вас назвать какой-либо номер (договора, телефона и пр.), по которому вас можно идентифицировать в системе.

Если вы забыли свой номер, то оператор службы технической поддержки попросит предоставить вас какую-либо другую информацию, которая поможет уникальным образом идентифицировать вас.

Например, комбинация вашего дня рождения и фамилия. Они тоже могут являться первичным ключом, точнее их комбинация.

Первичный ключ уникален. Первичный ключ всегда имеет уникальное значение. Представьте, что его значение не уникально. Тогда его бы нельзя было использовать для того, чтобы идентифицировать данные в таблице. Это значит, что какое-либо значение первичного ключа может встретиться в столбце, который выбран в качестве первичного ключа, только один раз. РСУБД устроены так, что не позволят вам вставить дубликаты в поле первичного ключа, получите ошибку.

Еще один пример. Представьте, что у вас есть таблица с полями `first_name` и `last_name` и есть две записи:

Свойства первичного ключа:

- уникальность — в таблице может быть назначен только один первичный ключ, у составного ключа поля могут повторяться, но не все;
- избыточность — не должно быть полей, которые, будучи удаленными из первичного ключа, не нарушат его уникальность;
- в состав первичного ключа не должны входить поля типа, комментариев и графическое.

Типы первичных ключей.

Обычно первичный ключ — числовое значение. Но он также может быть и любым другим типом данных. Не является обычной практикой использование строки в

качестве первичного ключа (строка – фрагмент текста), но теоретически и практически это возможно.

Составные первичные ключи. Часто первичный ключ состоит из одного поля, но он может быть и комбинацией нескольких столбцов, например, двух (трех, четырех...). Но помните, что первичный ключ всегда уникален, а значит нужно, чтобы комбинация n-го количества полей, в данном случае 2-х, была уникальна.

Поле первичного ключа часто, но не всегда, обрабатывается самой базой данных. Вы можете, условно говоря, сделать так, чтобы уникальное числовое значение каждой записи при ее создании присваивалось автоматически. Обычно, нумерация начинается с 1 и это число увеличивается для каждой записи на одну единицу. Такой первичный ключ называется автоинкрементным или автономерованным. Использование автоинкрементных ключей – хороший способ для задания уникальных первичных ключей. Классическое название такого ключа – суррогатный первичный ключ. Такой ключ не содержит полезной информации, относящейся к сущности (объекту), информация о которой хранится в таблице, поэтому он и называется суррогатным.

Связь – это осмысленная ассоциация между сущностями. Число сущностей, которое может быть ассоциировано через набор связей с другой сущностью, называют степенью связи. Связь между двумя сущностями называется бинарной, а связь между более чем с двумя сущностями – тернарной. На ER-диаграмме связь изображается ромбом.

Основное назначение связей - это возможность организации поиска данных в базе данных.

Важной характеристикой связи является тип связи (кардинальность)

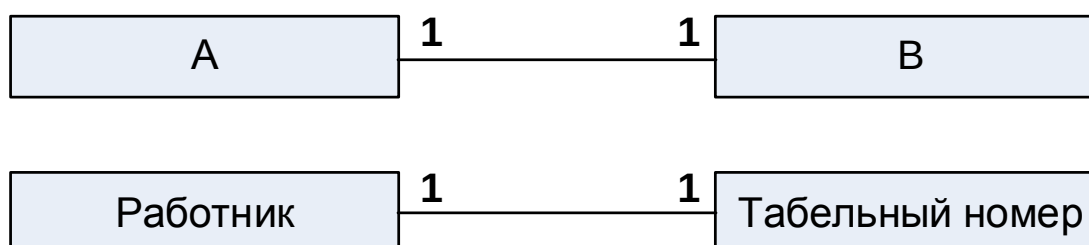
Для связи указывается название, кардинальность (1:1, 1:n или m:n) и степень (бинарная, тернарная).

Каждая связь имеет два конца и одно или два наименования. Наименование обычно выражается в неопределенной глагольной форме: "иметь", "принадлежать" и т.п. Каждое из наименований относится к своему концу связи. Иногда наименования не пишутся ввиду их очевидности.

Каждая связь может иметь один из следующих типов связи:

- один к одному
- один ко многим
- многие ко многим

Связь типа «**Один-к-одному**» означает, что любому экземпляру сущности А соответствует только один экземпляр сущности В, и наоборот.



Связь один-к-одному легко моделируется в одной таблице. Записи таблицы содержат данные, которые находятся в связи один-к-одному с первичным ключом.

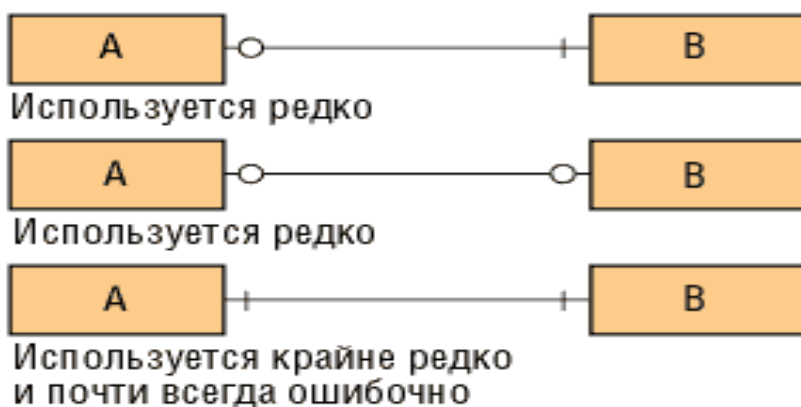
Обычно наличие двух таблиц в связи один-к-одному считается дурной практикой.

Связь один-к-одному чаще всего свидетельствует о том, что на самом деле мы имеем всего одну сущность, неправильно разделенную на две.

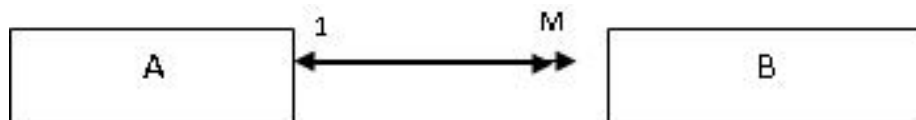
Отношение «один-к-одному» имеет место, когда одной записи в родительской таблице соответствует одна запись в дочерней таблице.

№ сотрудника	ФИО	Должность	Отдел		№ сотрудника	Год рождения	Число детей
1	Иванов П.П.	лаборант	2	→	1	1967	1
2	Сидорова А.М.	бухгалтер	1	→	2	1955	1
3	Петров А.Н.	инженер	2	→	3	1943	2

Допустимые типы связей "один- к- одному" показаны на рисунке ниже.



Связь типа «Один-ко-многим» означает, что любому экземпляру сущности А соответствует 0, 1 или несколько экземпляров сущности В, но любому экземпляру сущности В соответствует только один экземпляр сущности А.



Например, студенту ставят много отметок; поставленная отметка принадлежит только одному студенту.

Пример связи «один-ко-многим». Дочерняя и родительская таблицы связаны между собой по общему полю «Шифр группы»

Таблица «Группа»

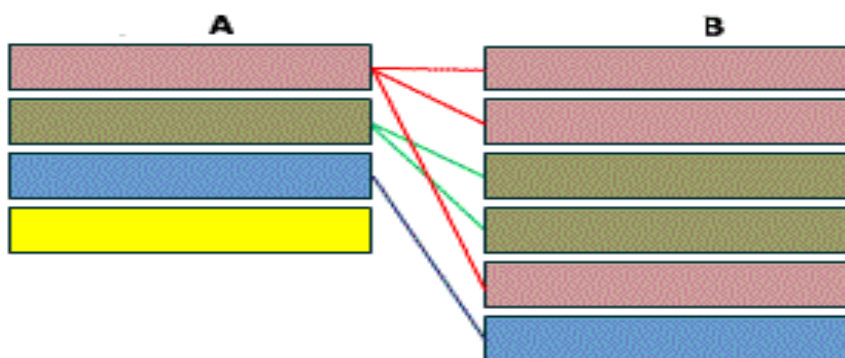
Шифр группы	Кол-во студ-в	Прох. балл
101	30	4,50
102	32	4,50
103	29	4,80

Таблица «Студент»

Шифр группы	№ студ.	ФИО	Год рожд.	Прох. балл
101	01	Аристов Р.П.	1979	4,25
101	02	Борисова С.А.	1979	4,25
102	01	Макова Н.В.	1978	4,75

Это наиболее часто используемый тип связи. Левая сущность (со стороны «один») называется родительской, правая (со стороны «много») - дочерней.

Как опознать связь один-ко-многим?



Если у вас есть две сущности спросите себя:

- 1) Сколько объектов из В могут относиться к объекту А?
- 2) Сколько объектов из А могут относиться к объекту В?

Если на первый вопрос ответ – **множество**, а на второй – **один** (или возможно, что ни одного), то вы имеете дело со связью один-ко-многим.

Примеры связи один-ко-многим:

- Машина и ее части

Каждая часть машины одновременно принадлежит только одной машине, но машина может иметь множество частей.

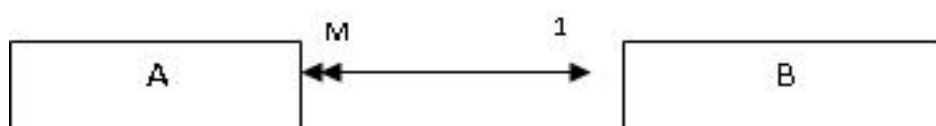
- Кинотеатры и экраны

В одном кинотеатре может быть множество экранов, но каждый экран принадлежит только одному кинотеатру.

- Дома и улицы

На улице может быть несколько домов, но каждый дом принадлежит только одной улице.

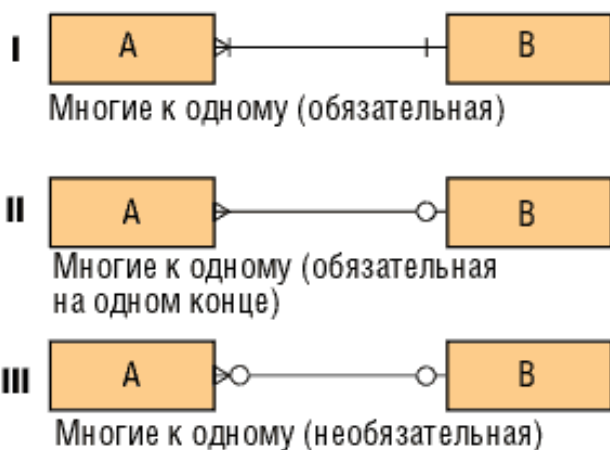
Связь типа «**Многие-к-одному**» означает, что любому экземпляру сущности А соответствует только один экземпляр сущности В, но любому экземпляру сущности В соответствует 0, 1 или несколько экземпляров сущности А.



Например, Преподаватель работает только в одном кабинете, однако рабочий кабинет может быть закреплен за несколькими преподавателями.

На рисунке ниже показаны допустимые связи типа «многие –к-одному»

- I - достаточно сильная конструкция, предполагающая, что вхождение сущности В не может быть создано без одновременного создания по меньшей мере одного связанного с ним вхождения сущности А.
- II - это часто встречающаяся форма связи. Предполагает, что каждое и любое вхождение сущности А может существовать только в контексте одного (и только одного) вхождения сущности В. В свою очередь, вхождения В могут существовать как в связи с вхождениями А, так и без нее.
- III - применяется редко. Как А, так и В могут существовать без связи между ними.

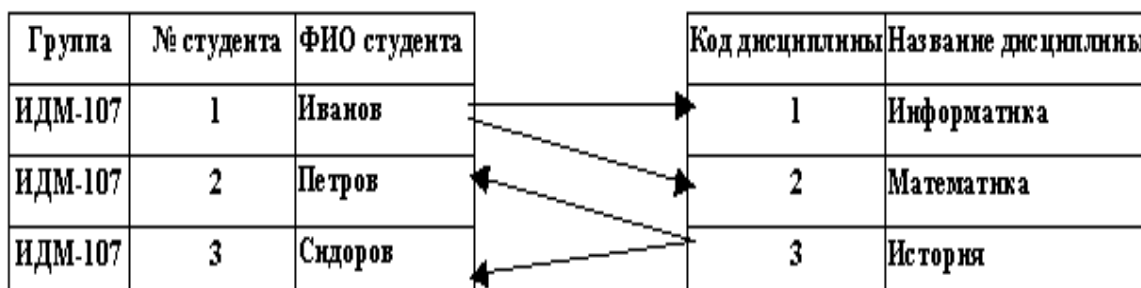


Связь типа «**Многие-ко-многим**» означает, что любому экземпляру сущности А соответствует 0, 1 или несколько экземпляров сущности В, и любому экземпляру сущности В соответствует 0, 1 или несколько экземпляров сущности А.

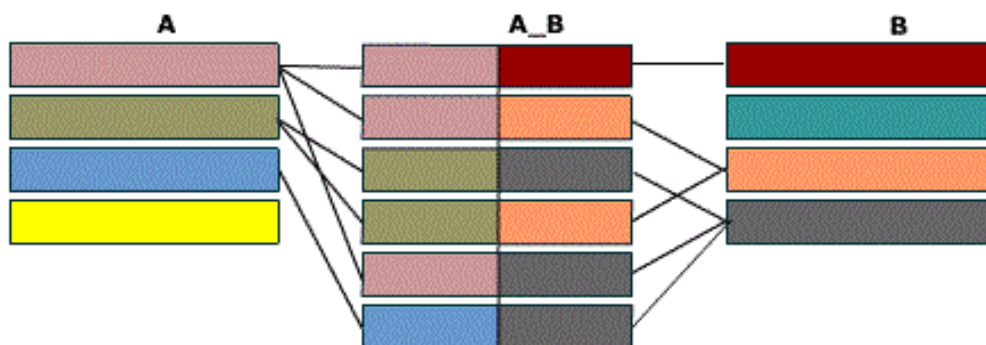
Тип связи много-ко-многим является временным типом связи, допустимым на ранних этапах разработки модели. В дальнейшем этот тип связи должен быть заменен двумя связями типа один-ко-многим путем создания промежуточной сущности.

Отношение «многие-ко-многим» имеет место, когда:

- записи в родительской таблице может соответствовать больше одной записи в дочерней таблице;
- записи в дочерней таблице может соответствовать больше одной записи в родительской таблице.



Связь многие-ко-многим создается с помощью трех таблиц. Две таблицы – “источника” и одна соединительная таблица. Первичный ключ соединительной таблицы A_B – составной. Она состоит из двух полей, двух внешних ключей, которые ссылаются на первичные ключи таблиц A и B. Это показано на рисунке ниже.

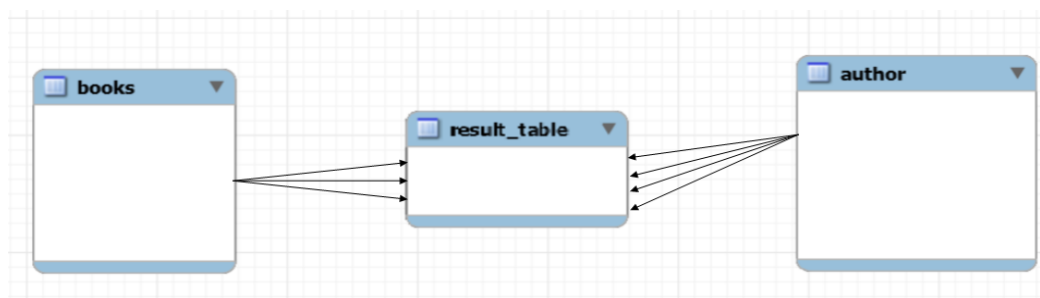


Например, пусть имеется таблица с книгами и таблица с авторами. Приведем два верных утверждения.

Первое: одну книгу может написать несколько авторов.

Второе: автор может написать несколько книг.

Это типичный пример связи многие ко многим. Такая связь (связь многие ко многим) реализуется путем добавления третьей таблицы.



Одна книга могла быть написана несколькими авторами.

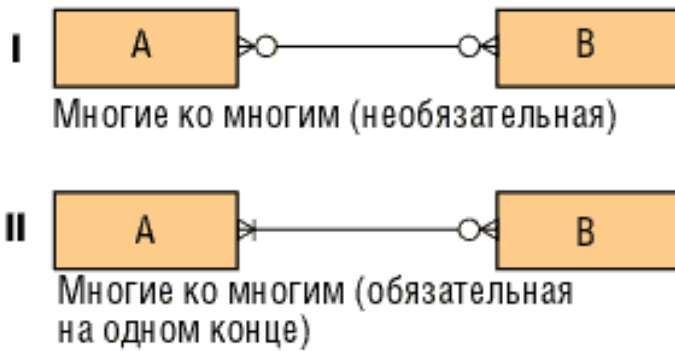
Автор мог написать несколько книг.

Допустимые связи "многие-ко-многим" показаны на рисунке ниже.

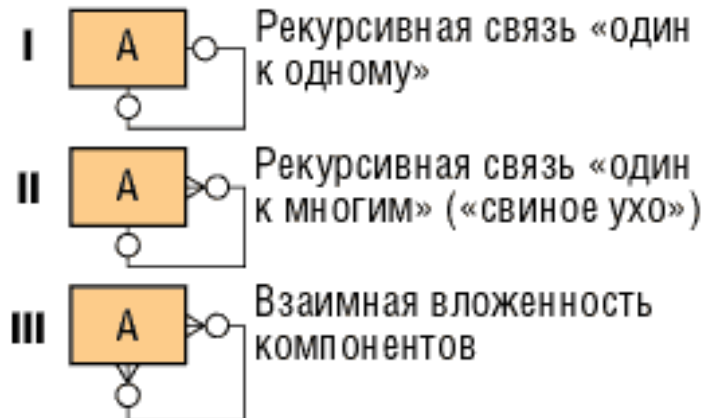
Здесь:

- I - такая конструкция часто имеет место в начале этапа анализа и означает связь - либо понятую не до конца и требующую дополнительного разрешения, либо отражающую простое коллективное отношение - двунаправленный список.

- II - применяется редко. Такие связи всегда подлежат дальнейшей детализации.



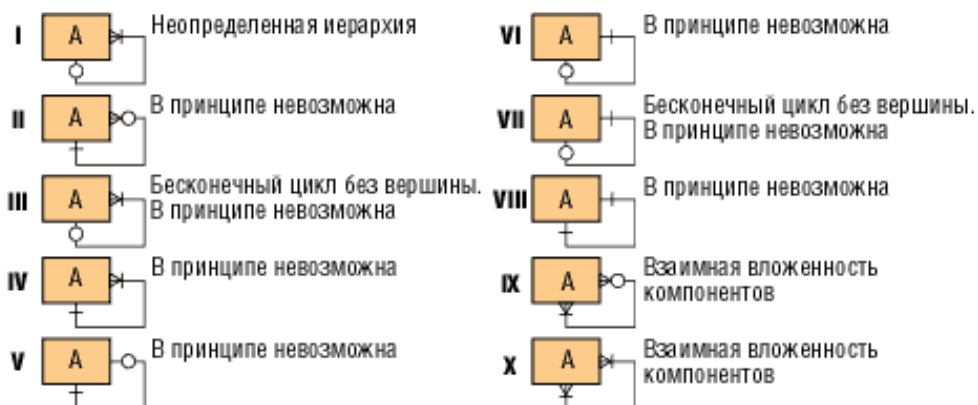
На рисунке ниже показаны варианты рекурсивных связей.



К недопустимым типам связей относятся следующие: обязательная связь "многие ко многим" как показано на рисунке ниже.



и ряд рекурсивных связей, изображенных на рисунке ниже.



Каждая связь может иметь одну из двух модальностей связи.

Модальность «может» означает, что экземпляр одной сущности может быть связан с одним или несколькими экземплярами другой сущности, а может быть и не связан ни с одним экземпляром.

Модальность «должен» означает, что экземпляр одной сущности обязан быть связан не менее чем с одним экземпляром другой сущности.

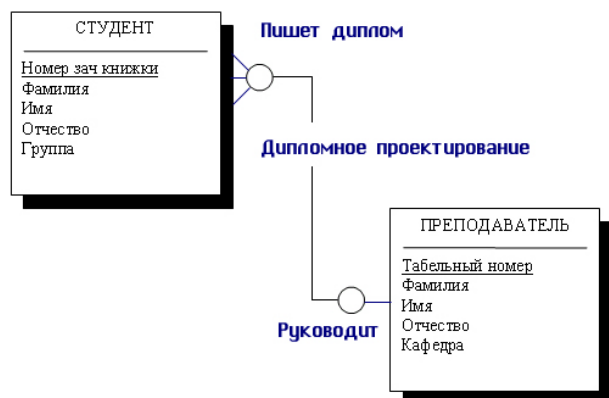
Связь может иметь разную модальность с разных концов.

Описанный графический синтаксис позволяет однозначно читать диаграммы, пользуясь следующей схемой построения фраз:

<Каждый экземпляр СУЩНОСТИ 1> <МОДАЛЬНОСТЬ СВЯЗИ>
<НАИМЕНОВАНИЕ СВЯЗИ> <ТИП СВЯЗИ> <экземпляр СУЩНОСТИ 2>.

Каждая связь может быть прочитана как слева направо, так и справа налево.

Например, если у нас есть связь между сущностью «Студент» и сущностью «Преподаватель» и эта связь — руководство дипломными проектами, то каждый студент имеет только одного руководителя, но один и тот же преподаватель может руководить множеством студентов-дипломников, как показано на рисунке ниже.



Например, как показано на рисунке ниже, каждый студент, который пишет диплом, должен иметь своего руководителя дипломного проектирования, но, с другой стороны, не каждый преподаватель должен вести дипломное проектирование.

